ALPHA COUNCIL v2.4

Competition-Grade Implementation Specification - Dynamic-Discovery Edition

Alpaca AI Trading Agents Hackathon - August 28 to September 4, 2026

Item	Value
Submission deadline	**September 4, 2026, 15:00 UTC (11:00 ET)**
Last full trading session	**September 3, 2026**
Trading sessions available	Aug 28, Aug 31, Sep 1, Sep 2, Sep 3
Target environment	Windows 11, Python 3.11, VS Code
Coding agents	Claude Code (primary), OpenAI Codex (secondary)
Operator time	2-4 hours/day, agent-assisted
Execution venue	Alpaca competition paper account (Level 3 options enabled)
Market data plan	**Alpaca Basic (free): IEX equities real-time; options use Indicative Pricing Feed; latest-15-minute historical restriction applies**
AI budget	$50 OpenAI API + $50 Anthropic API

This document supersedes Alpha Council v2.3 in full. Where v2.3 and v2.4 conflict, v2.4 governs. The current codebase is already partially implemented from v2.3; Section 31 is the mandatory migration patch and MUST be applied without rebuilding completed components unnecessarily.

v2.4 change summary

v2.4 preserves the v2.3 architecture and differentiator, but changes opportunity discovery and execution calibration:

1. Keep the existing ~65-name list as the **Core Universe**, not the maximum search universe.
2. Add a **Dynamic Discovery Universe** capped at 200-250 symbols, with staged filtering before options or LLM work.
3. Restore Alpaca most-active and market-mover discovery as opportunistic sources; a 403 disables that source without degrading the core system.
4. Allow Alpaca News and, when practical, SEC current-filings discovery to inject off-core symbols for temporary evaluation.
5. Use the funnel **~200-250 -> 30 -> 12 -> 5 -> up to 3 councils per scan**.
6. When the system lacks trades, **expand breadth before lowering quality thresholds**.
7. Tier 3 MUST retain meaningful options liquidity; zero-volume legs and extremely wide spreads are not an acceptable anti-zero-trade mechanism.
8. Treat the six-trade goal as **lifecycle demonstration coverage**, not a mandate to force six alpha bets.
9. Correct the data assumption: Alpaca Basic options use the **Indicative Pricing Feed**. The latest-15-minute restriction is a historical-data limitation; indicative quotes are derived/modified rather than OPRA NBBO, and trades may be delayed. Measured timestamps and fill behavior govern calibration.
10. Add an **Execution/Fill Calibration Engine** that measures indicative-reference-to-fill bias and limit-walk behavior.
11. Schwab remains outside the MVP. It MAY be added post-MVP as a warning-only validator for the final selected two-leg spread, never as an execution dependency or hard gate.
12. Add two candidate paths: **EVENT** and **MOMENTUM**, both converging on the same Council, Red Team, Risk Constitution, and Alpaca execution path.

---

0. How Claude Code / Codex Should Use This Document

This is a normative implementation specification. MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are intentional.

1. Build/patch in the order given in §22 and §31. The first operational milestone remains a **working end-to-end paper trade**, not a complete scanner.
2. Do not replace deterministic rules with LLM prompts.
3. Do not let any LLM invent option contracts, prices, account state, fills, universe membership facts, or risk limits.
4. Alpaca is the only broker and the only execution venue.
5. Every AI response consumed by code MUST parse through a Pydantic model or provider-native structured-output schema.

6. Every trade decision MUST be reconstructable from the SQLite audit trail, including the configuration version and discovery source in force at decision time.
7. If a required input is stale beyond its configured tolerance, missing, contradictory, or fails schema validation, the default is NO TRADE - and the reason MUST be written to gate_rejections.
8. Do not introduce LangChain, CrewAI, AutoGen, Kafka, Kubernetes, Redis, Postgres, or a microservice architecture during the competition build.
9. **Breadth-before-looseness rule:** if no trade qualifies, expand the deterministic discovery set before relaxing quality thresholds. Hard safety gates never move.
10. Do not rebuild completed v2.3 components merely because this document is newer. Apply the explicit migration patch in §31 to the existing build.
11. Any optional data source (movers, most-actives, Schwab post-MVP validation) MUST fail open for discovery/validation and MUST NOT halt trading infrastructure.

1. Objective, Priorities, and Differentiator

Alpha Council is an autonomous, options-native, evidence-driven trading desk. Deterministic Python finds candidates and constructs contracts. LLMs reason about evidence. Deterministic code decides what is permitted.

AI decides what it wants to do. Deterministic software decides what it is allowed to do.

1.1 Priority order

1. Win the hackathon.
2. Produce a defensible, auditable decision record with real paper fills.
3. Generate positive P&L across a small sample, acknowledging that 5 sessions of results is anecdote, not evidence.
4. Preserve a path to a longer-lived portfolio product.

1.2 The differentiator

A council of arguing agents is **not** the differentiator - other teams in this hackathon are building the same bull/bear/risk debate structure. The differentiator is **Counterfactual Decision Attribution plus Gate Attribution**:

- Every trade keeps shadow variants (GPT original / Claude modified / Executed) and measures each layer's P&L contribution, split into **per-spread effect** (structure selection) and **sizing effect** (risk sizing).
- Every *rejected* candidate is also shadow-marked, so the system can state what each individual gate cost or saved.

No other team will be able to say "our risk engine added $310 and our red team cost us $240, and here is the arithmetic."

1.3 MVP strategy set

Only two structures. Both are two-leg defined-risk debit verticals:

- **Bull call debit spread** for bullish views.
- **Bear put debit spread** for bearish views.

Credit spreads, calendars, condors, 0DTE, and naked options are out of scope for the competition.

1.4 Trading universe

The existing seed list becomes the **Core Universe**. It is always scanned and remains the preferred pool for reliable IEX density and liquid options.

SPY QQQ IWM DIA
AAPL MSFT NVDA AMZN META GOOGL TSLA AVGO AMD MU INTC QCOM ARM
NFLX ORCL CRM ADBE NOW PLTR IBM CSCO PANW CRWD SNOW DELL
JPM BAC WFC GS MS C SCHW COIN HOOD SOFI
XOM CVX COP SLB
LLY UNH JNJ ABBV MRK PFE
WMT COST HD LOW NKE SBUX MCD
CAT DE GE RTX BA UPS
DIS UBER ABNB BKNG

The Core Universe is not the search ceiling. v2.4 adds a Dynamic Discovery Universe:

Layer	Target size	Purpose
Core Universe	~65	Always monitored; stable/liquid baseline
Dynamic Discovery Universe	max 200-250	Cheap deterministic opportunity discovery
Stage-1 survivors	top 30	Full quant feature computation
Options pre-screen	top 12	Fetch/inspect chains only where justified
Final candidate set	top 5	Final Opportunity Score and evidence pack
Councils	up to 3/scan (Tier 1/2), 4 at Tier 3	Expensive LLM reasoning

Dynamic candidates may come from:

1. Core Universe.
2. Active Alpaca assets with `has_options`, filtered for tradability and price/data quality.
3. Alpaca **most-active** screener if the account is entitled.
4. Alpaca **market-movers** screener if the account is entitled.
5. Symbols attached to fresh Alpaca News items.
6. Symbols mapped from fresh SEC filings when a global/current-filings discovery feed is available.

Any dynamic symbol MUST pass asset eligibility, IEX data-density, price, directional-ambiguity, and options-availability checks before options-chain work. Dynamic membership is temporary (default TTL 90 minutes) unless the symbol also belongs to Core.

If an optional screener returns HTTP 403, record `DISCOVERY_SOURCE_FORBIDDEN`, disable that source for the session, and continue with the remaining sources. Do not retry a forbidden source every scan.

1.5 Activity target

The system SHOULD demonstrate **at least six complete order lifecycles** across the competition, but it MUST NOT force six alpha bets.

Separate two concepts:

- **Calibration / engineering lifecycle trades:** 2-3 very small, supervised, liquid-spread tests used to validate execution, fill calibration, opening/closing, idempotency, and journaling.
- **Alpha trades:** only trades that qualify through the normal discovery, Council, Red Team, and Risk Constitution path.

A realistic competition target is **3-8 genuine alpha trades** plus calibration lifecycle trades as needed. More than ~20 genuine alpha entries suggests filters are too loose. Zero genuine alpha trades is undesirable, but the response is to expand search breadth first (§12.6), not to accept structurally poor options.

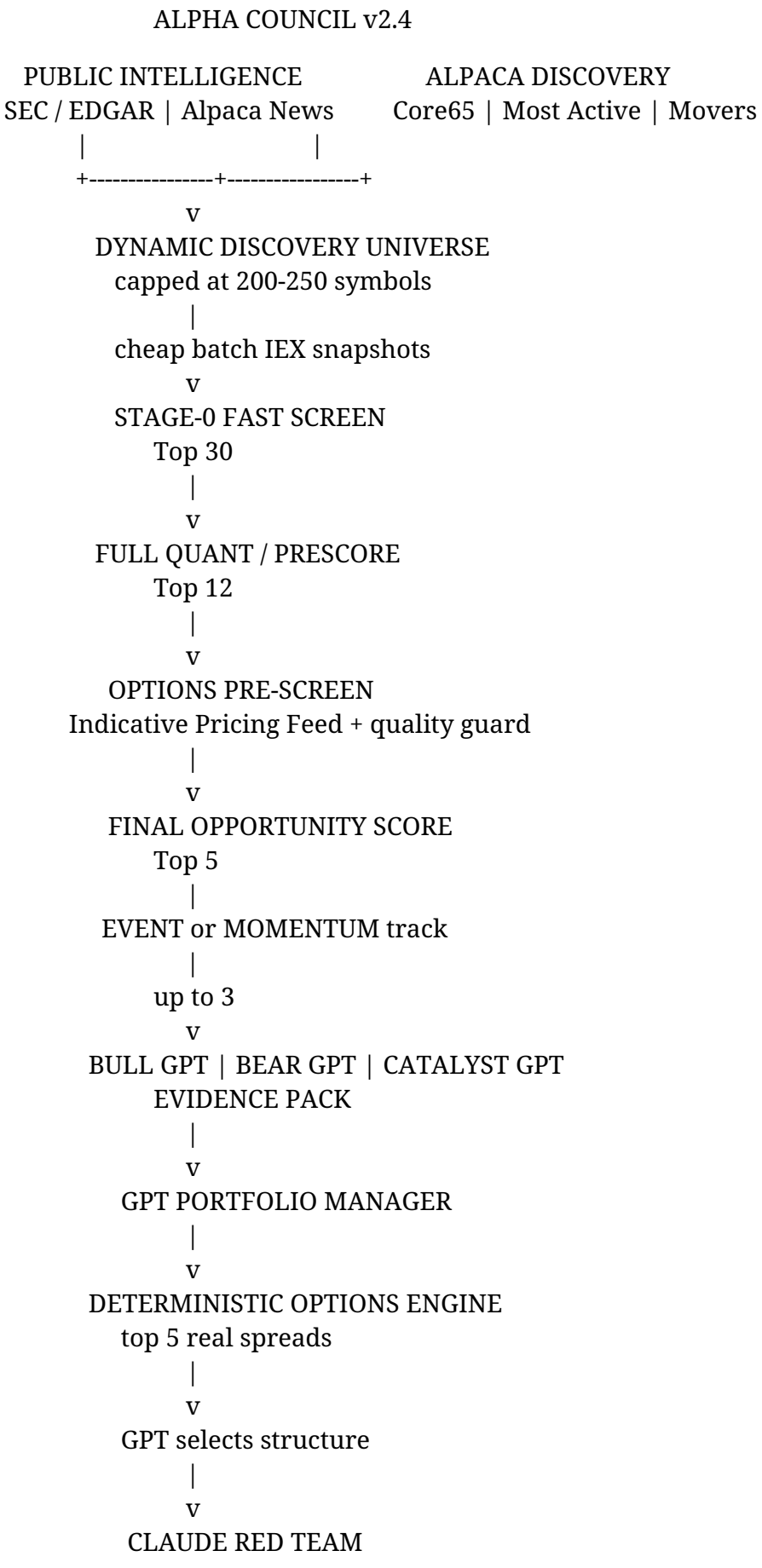
---

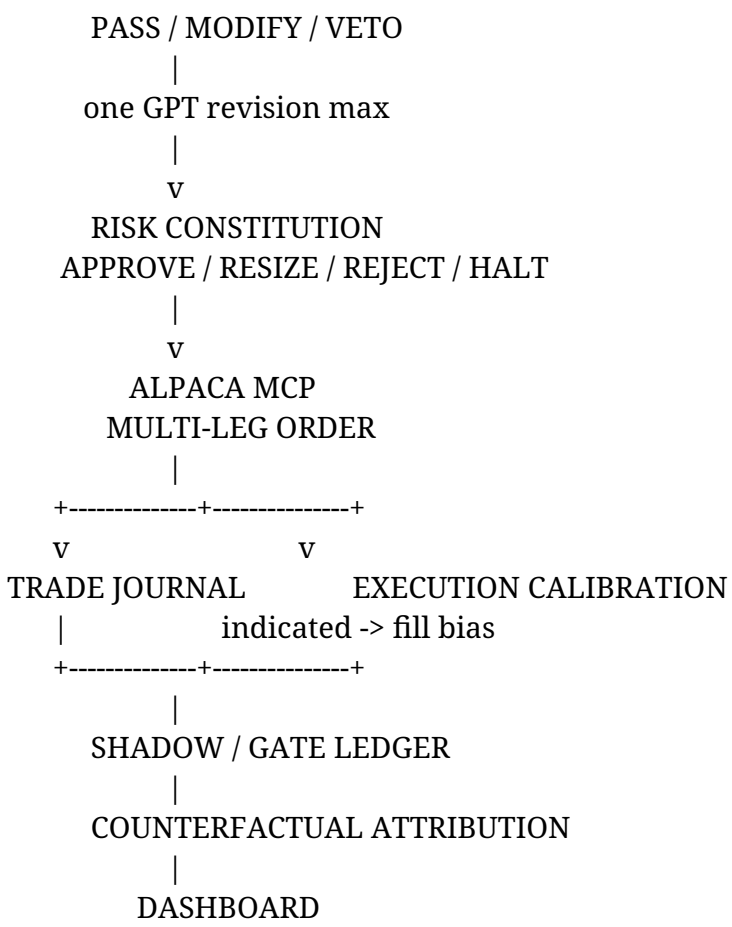
2. Non-Negotiable Architecture Invariants

1. **Alpaca is execution authority.** All orders and positions come from the dedicated competition paper account.
2. **Paper-only hard lock.** The process MUST refuse to start if `ALPACA_PAPER_TRADE` is not `true` or if the account reports live status.
3. **No LLM market-wide scanning.** Python scans; LLMs only see candidates that already passed quantitative gates.
4. **No LLM strike invention.** The options engine returns real contracts from live chain data.
5. **No LLM risk enforcement.** Limits are code.
6. **Option quotes are assumed stale.** All option pricing logic MUST carry an explicit `quote_lag_seconds` and MUST apply the stale-quote adjustment in §5.4 before using a mark for a decision.
7. **Exits are driven by the underlying, not the option.** Equity data is real-time; option data is not. Intraday invalidation and stop logic MUST be computable from underlying price/VWAP alone.
8. **No silent fallbacks.** Missing data produces an explicit degraded state.
9. **No duplicate order retries.** Unique `client_order_id`; after any timeout, query by client ID before retrying.
10. **Every decision and every rejection is journaled**, with the active `config_version`.
11. **Hard safety gates are never auto-relaxed.** Only quality gates participate in the adaptive ladder (§12.6).

---

3. System Architecture





3.1 Control plane vs data plane

Use **Alpaca MCP V2** for account state, positions, option contract inspection, order lifecycle, and execution. Use **direct Alpaca REST** for high-throughput batch scanning (snapshots, bars, news, optional screeners) behind the same `alpaca` adapter package. MCP remains central to the competition story; REST handles volume.

3.2 Discovery vs deep analysis

The system MUST NOT fetch full options chains or invoke LLMs for the entire Dynamic Discovery Universe. The competitive advantage comes from broad, cheap discovery followed by aggressive narrowing:

```
200-250 discovery symbols
  -> top 30 fast-screen survivors
  -> top 12 full prescore survivors
  -> top 5 options-qualified candidates
  -> up to 3 councils per normal scan
```

This preserves breadth without multiplying the expensive parts of the pipeline.

4. Technology Stack

Layer	Choice
OS / Language	Windows 11, Python 3.11
Package manager	uv
Schemas	Pydantic v2
HTTP / async	httpx, asyncio, tenacity
Database	SQLite + aiosqlite
Scheduling	APScheduler
Numerics	pandas, numpy
Dedupe	RapidFuzz
Parsing	feedparser, BeautifulSoup4, lxml
OpenAI	Responses API + Structured Outputs
Anthropic	Messages API + Structured Outputs
Alpaca	MCP V2 + REST

Layer	Choice
Dashboard	Streamlit + Plotly
Tests	pytest, pytest-asyncio, respx

```
uv add pydantic pydantic-settings httpx tenacity aiosqlite
uv add pandas numpy rapidfuzz
uv add feedparser beautifulsoup4 lxml
uv add openai anthropic mcp
uv add apscheduler python-dotenv
uv add streamlit plotly
uv add pytest pytest-asyncio respx --dev
```

schwab-py, scipy, and the Schwab package tree are removed (see §28).

4.1 Environment variables

```
APP_ENV=development
TIMEZONE=America/New_York
DATABASE_PATH=./data/alpha_council.db

ALPACA_API_KEY=
ALPACA_SECRET_KEY=
ALPACA_PAPER_TRADE=true
ALPACA_DATA_FEED=iex
ALPACA_OPTION_FEED=indicative

OPENAI_API_KEY=
ANTHROPIC_API_KEY=

SEC_USER_AGENT=AlphaCouncil/0.1 operator@example.com

CONFIG_VERSION=v1
```

5. Free-Plan Data Reality - Corrected Interpretation

Alpaca Basic provides real-time equities coverage from **IEX** and options coverage from the **Indicative Pricing Feed**. The Basic plan also has a **latest-15-minute historical-data restriction**. Do not conflate that historical restriction with the timestamp age of every latest indicative quote.

For options, the Indicative feed is a free derivative of OPRA: indicative quotes are **not OPRA NBBO quotes**, and indicative trades may be delayed. Therefore the key risk is not merely quote age; it is **quote fidelity plus timestamp freshness**.

5.1 What this means for Alpha Council

Data issue	v2.4 handling
IEX equities are only one venue	Use same-feed ratios (especially RVOL), not consolidated absolute-volume comparisons
Indicative option quote timestamp may be fresh	Measure timestamp age; do not assume 15-minute age
Indicative quote value is not OPRA NBBO	Treat as an estimate/reference, not executable truth
Indicative option trades may be delayed	Do not infer current option price action from trade prints
Historical latest-15-minute restriction	Avoid depending on unavailable historical near-now windows
Option pricing uncertainty	Use timestamp/drift checks + conservative limit logic + measured fill calibration (§17.5)
Intraday exit monitoring	Drive exits from the underlying, not option marks

5.2 Mandatory live-session probe

Before locking the options engine thresholds, run scripts/probe_data_reality.py during a normal RTH session. It MUST:

1. Fetch the SPY indicative option chain/snapshots and record quote timestamps for at least 10 liquid contracts; repeat over at least 10 minutes.

2. Fetch IEX stock snapshots for SPY and NVDA and record quote/trade/bar timestamp lags.
3. Confirm Greeks and implied volatility are present.
4. Confirm open_interest and open_interest_date; treat OI as prior-session information.
5. Record bid/ask validity and instances of zero/invalid asks.
6. Persist the observed lag distribution to system_events and print a one-page summary.

Set options.fresh_quote_seconds and options.max_quote_lag_seconds from measured behavior, not from a blanket 15-minute assumption. The already-observed ~1.5-second in-session sample is encouraging but insufficient; Monday's probe decides the configuration.

5.3 Demo narrative

Describe the limitation precisely:

> Alpha Council uses Alpaca's free Indicative Pricing Feed for options. It treats indicative quotes as derived estimates rather than OPRA NBBO, measures their timestamp freshness, adjusts for underlying movement when needed, executes with conservative limits, and calibrates the indicated-to-fill bias from actual Alpaca paper fills.

This is a stronger engineering claim than calling every quote "15 minutes delayed."

5.4 Indicative-Quote Delta Adjustment

When an indicative option quote is older than options.fresh_quote_seconds, the raw mid MUST NOT be treated as current. Compute:

```
underlying_move = underlying_price_now - underlying_price_at_option_quote_time
adjusted_leg_mid = raw_leg_mid + (leg_delta * underlying_move)
adjusted_spread_mid = adjusted_long_mid - adjusted_short_mid
```

Rules:

- underlying_price_at_option_quote_time MUST come from stored IEX bars/quotes at that timestamp.
- If abs(underlying_move / underlying_price) > options.max_underlying_drift_pct, mark the structure STALE_UNUSABLE and reject it with OPT_STALE_DRIFT.
- Gamma is ignored deliberately; document this approximation.
- Record raw_mid, adjusted_mid, quote_lag_seconds, and underlying_move on every option structure and shadow mark.
- Even when timestamp freshness is HIGH, an Indicative quote remains non-OPRA and is still subject to execution calibration.
- Use the same marking method across GPT-original, Claude-modified, executed, and rejected-shadow variants.

6. Repository Layout

```
alpha-council/
|-- README.md
|-- pyproject.toml
|-- .env.example
|-- config/
|   |-- universe.yaml
|   |-- risk_constitution.yaml
|   |-- scoring.yaml
|   |-- osint_sources.yaml
|   |-- event_calendar.yaml
|   `-- prompts/
|       |-- bull_system.txt
|       |-- bear_system.txt
|       |-- catalyst_system.txt
```

```
|       |-- pm_system.txt
|       |-- pm_revision_system.txt
|       `-- red_team_system.txt
|
|-- alpha_council/
|   |-- settings.py
|   |-- logging_config.py
|   |-- models/       (enums, market, intelligence, candidate, trading, risk, execution)
|   |-- db/           (schema.sql, engine.py, repositories.py)
|   |-- utils/        (time, ids, hashing, retry, math)
|   |-- alpaca/       (mcp_client, rest_client, market_data, news, options, account, execution)
|   |-- data/         (quality.py, staleness.py)
|   |-- intelligence/  (registry, normalizer, deduplicator, scoring, entity_mapper,
|   |                   event_builder, sources/{sec.py, alpaca_news.py})
|   |-- quant/        (indicators, regime, features, scoring, scanner, discovery)
|   |-- options_engine/(chain, liquidity, spreads, scoring)
|   |-- agents/       (openai_client, anthropic_client, bull, bear, catalyst,
|   |                  portfolio_manager, red_team, budget.py)
|   |-- risk/         (constitution, position_sizing, event_blackout, gates.py)
|   |-- journal/      (trade_journal, shadow_book, attribution, rejection_log.py)
|   |-- execution/    (order_builder, order_manager, position_monitor, fill_calibration)
|   |-- orchestrator.py
|   `-- scheduler.py
|
|-- dashboard/app.py
|-- scripts/
|   |-- probe_data_reality.py
|   |-- init_db.py
|   |-- test_alpaca_mcp.py
|   |-- vertical_slice.py
|   |-- scan_once.py
|   |-- council_once.py
|   |-- dry_run_trade.py
|   |-- gate_report.py
|   |-- close_all.py
|   `-- run_alpha_council.py
`-- tests/
```

alpha_council/schwab/ and alpha_council/replay.py are removed. Replay is achieved by re-running the orchestrator against stored database rows (§20.3).

---

7. Domain Models - Changes from v2.2

All v2.2 models are retained with the following modifications. StrictModel (extra="forbid", validate_assignment=True) remains the base.

7.1 Replaced: ConsensusResult → DataQualityResult

```python
class DataConfidence(StrEnum):
    HIGH = "HIGH"
```

```
    MEDIUM = "MEDIUM"
    DEGRADED = "DEGRADED"
    BLOCKED = "BLOCKED"

class DataQualityResult(StrictModel):
    symbol: str
    asset_type: Literal["EQUITY", "ETF", "OPTION"]
    evaluated_at: datetime
    source: Literal["ALPACA_IEX", "ALPACA_INDICATIVE"]
    quote_timestamp: datetime | None = None
    quote_lag_seconds: float | None = Field(default=None, ge=0)
    bid: float | None = None
    ask: float | None = None
    raw_mid: float | None = None
    adjusted_mid: float | None = None
    underlying_move: float | None = None
    spread_pct: float | None = None
    confidence: DataConfidence
    confidence_factor: float = Field(ge=0, le=1)
    signal_price: float | None = None
    execution_reference_price: float | None = None
    reason: str
```

7.2 OptionLeg - added fields

```
    raw_mid: float = Field(ge=0)
    adjusted_mid: float = Field(ge=0)
    quote_lag_seconds: float = Field(ge=0)
    underlying_price_at_quote: float | None = None
    open_interest_date: date | None = None
```

7.3 OptionStructure - added fields

```
    net_delta: float
    cost_to_width_ratio: float = Field(gt=0, le=1)
    staleness_buffer: float = Field(ge=0)
    max_quote_lag_seconds: float = Field(ge=0)
    stale_adjusted: bool = False
```

`rank` becomes `Field(ge=1, le=5)` - the engine now returns up to five structures so the PM has real choice when the chain is thin.

7.4 New: GateRejection

```
class GateStage(StrEnum):
    UNIVERSE = "UNIVERSE"
    DATA_QUALITY = "DATA_QUALITY"
    PRESCORE = "PRESCORE"
    OPTIONS_CHAIN = "OPTIONS_CHAIN"
    OPTIONS_STRUCTURE = "OPTIONS_STRUCTURE"
    OPPORTUNITY_SCORE = "OPPORTUNITY_SCORE"
    BUDGET = "BUDGET"
```

```
    PM_ABSTAIN = "PM_ABSTAIN"
    RED_TEAM = "RED_TEAM"
    RISK = "RISK"
    EXECUTION = "EXECUTION"

class GateRejection(StrictModel):
    rejection_id: str
    occurred_at: datetime
    config_version: str
    scan_id: str | None = None
    decision_id: str | None = None
    symbol: str
    direction: Direction
    stage: GateStage
    gate_id: str
    observed_value: float | str | None = None
    threshold_value: float | str | None = None
    tier: int = Field(ge=1, le=3)
    hard_gate: bool
    shadow_eligible: bool
    shadow_structure_json: str | None = None
    note: str
```

7.5 AttributionSnapshot - split effects

```
class AttributionSnapshot(StrictModel):
    decision_id: str
    as_of: datetime
    gpt_original_pnl: float
    claude_modified_pnl: float
    executed_pnl: float
    gpt_original_pnl_per_spread: float
    claude_modified_pnl_per_spread: float
    executed_pnl_per_spread: float
    gpt_original_qty: int
    claude_modified_qty: int
    executed_qty: int
    claude_selection_effect: float     # per-spread delta x executed qty
    claude_sizing_effect: float        # qty delta x claude per-spread pnl
    risk_selection_effect: float
    risk_sizing_effect: float
    claude_value_added: float
    risk_constitution_value_added: float
    mark_method: str
```

Attribution decomposition (required arithmetic):

```
selection_effect(A -> B) = (pnl_per_spread_B - pnl_per_spread_A) * qty_A
sizing_effect(A -> B)    = (qty_B - qty_A) * pnl_per_spread_B
total_effect(A -> B)     = selection_effect + sizing_effect
                         = (pnl_per_spread_B * qty_B) - (pnl_per_spread_A * qty_A)
```

This decomposition is exact and is the core demo artifact. It answers “did the red team pick a worse trade, or just a smaller one?”

7.6 New: discovery and track models

```
class DiscoverySource(StrEnum):
    CORE = "CORE"
    ALPACA_NEWS = "ALPACA_NEWS"
    SEC_EVENT = "SEC_EVENT"
    MOST_ACTIVE = "MOST_ACTIVE"
    MOVER = "MOVER"
    OTHER_DYNAMIC = "OTHER_DYNAMIC"

class CandidateTrack(StrEnum):
    EVENT = "EVENT"
    MOMENTUM = "MOMENTUM"
    CALIBRATION = "CALIBRATION"

class DiscoveryCandidate(StrictModel):
    symbol: str
    discovered_at: datetime
    expires_at: datetime | None = None
    source: DiscoverySource
    source_rank: int | None = None
    discovery_reason: str
    is_core: bool
    asset_tradable: bool
    has_options: bool
    data_density_ok: bool
    fast_score: float = Field(ge=0, le=100)

class FunnelSnapshot(StrictModel):
    scan_id: str
    as_of: datetime
    discovery_count: int
    stage0_survivors: int
    prescore_survivors: int
    options_prescreened: int
    final_candidates: int
    councils_started: int
```

7.7 New: execution calibration model

```
class ExecutionCalibration(StrictModel):
    calibration_id: str
    decision_id: str
    symbol: str
    side: Literal["OPEN", "CLOSE"]
    candidate_track: CandidateTrack
    submitted_at: datetime
    filled_at: datetime | None = None
```

```
    indicative_raw_mid: float
    indicative_adjusted_mid: float
    natural_debit_estimate: float
    initial_limit_debit: float
    final_submitted_limit: float
    actual_fill_debit: float | None = None
    seconds_to_fill: float | None = Field(default=None, ge=0)
    limit_walk_steps: int = Field(default=0, ge=0)
    quote_lag_seconds: float = Field(ge=0)
    underlying_at_quote: float
    underlying_at_submit: float
    underlying_at_fill: float | None = None
    fill_bias_vs_adjusted: float | None = None
    fill_bias_vs_limit: float | None = None
    fill_slippage_pct: float | None = None
```

---

8. Database Schema - Changes from v2.2

Retain the v2.2 schema with these modifications.

8.1 Bug fix: intelligence deduplication

```
-- v2.2 (WRONG): UNIQUE(content_hash)
-- A second independent source publishing identical text was rejected at insert,
-- making corroboration_score structurally incapable of exceeding one cluster.
-- v2.3:
    UNIQUE(source_id, source_native_id),
    UNIQUE(source_id, content_hash)
```

Cross-source identical content is now inserted and grouped by `duplicate_cluster_id` in the deduplicator, where corroboration counting belongs.

8.2 Replaced table

```
DROP TABLE IF EXISTS data_consensus;

CREATE TABLE IF NOT EXISTS data_quality (
    quality_id TEXT PRIMARY KEY,
    symbol TEXT NOT NULL,
    asset_type TEXT NOT NULL,
    evaluated_at TEXT NOT NULL,
    source TEXT NOT NULL,
    quote_timestamp TEXT,
    quote_lag_seconds REAL,
    bid REAL, ask REAL,
    raw_mid REAL,
    adjusted_mid REAL,
    underlying_move REAL,
    spread_pct REAL,
    confidence TEXT NOT NULL,
    confidence_factor REAL NOT NULL,
    signal_price REAL,
```

```
    execution_reference_price REAL,
    reason TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_quality_symbol_time
    ON data_quality(symbol, evaluated_at DESC);
```

8.3 New tables

```
CREATE TABLE IF NOT EXISTS config_versions (
    config_version TEXT PRIMARY KEY,
    activated_at TEXT NOT NULL,
    deactivated_at TEXT,
    scoring_json TEXT NOT NULL,
    risk_json TEXT NOT NULL,
    tier INTEGER NOT NULL,
    note TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS gate_rejections (
    rejection_id TEXT PRIMARY KEY,
    occurred_at TEXT NOT NULL,
    config_version TEXT NOT NULL REFERENCES config_versions(config_version),
    scan_id TEXT,
    decision_id TEXT,
    symbol TEXT NOT NULL,
    direction TEXT NOT NULL,
    stage TEXT NOT NULL,
    gate_id TEXT NOT NULL,
    observed_value TEXT,
    threshold_value TEXT,
    tier INTEGER NOT NULL,
    hard_gate INTEGER NOT NULL CHECK(hard_gate IN (0,1)),
    shadow_eligible INTEGER NOT NULL CHECK(shadow_eligible IN (0,1)),
    shadow_structure_json TEXT,
    note TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_gate_rejections_gate
    ON gate_rejections(gate_id, occurred_at DESC);
CREATE INDEX IF NOT EXISTS idx_gate_rejections_symbol
    ON gate_rejections(symbol, occurred_at DESC);

CREATE TABLE IF NOT EXISTS rejected_shadows (
    rejected_shadow_id TEXT PRIMARY KEY,
    rejection_id TEXT NOT NULL REFERENCES gate_rejections(rejection_id),
    structure_json TEXT NOT NULL,
    entry_timestamp TEXT NOT NULL,
    entry_reference_debit REAL NOT NULL,
    horizon_end TEXT NOT NULL,
    status TEXT NOT NULL,
```

```
    final_pnl_per_spread REAL,
    mark_method TEXT
);
```

8.3A New v2.4 discovery and execution-calibration tables

```
CREATE TABLE IF NOT EXISTS discovery_candidates (
    discovery_id TEXT PRIMARY KEY,
    scan_id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    discovered_at TEXT NOT NULL,
    expires_at TEXT,
    source TEXT NOT NULL,
    source_rank INTEGER,
    discovery_reason TEXT NOT NULL,
    is_core INTEGER NOT NULL CHECK(is_core IN (0,1)),
    asset_tradable INTEGER NOT NULL CHECK(asset_tradable IN (0,1)),
    has_options INTEGER NOT NULL CHECK(has_options IN (0,1)),
    data_density_ok INTEGER NOT NULL CHECK(data_density_ok IN (0,1)),
    fast_score REAL NOT NULL,
    UNIQUE(scan_id, symbol, source)
);
CREATE INDEX IF NOT EXISTS idx_discovery_scan_score
    ON discovery_candidates(scan_id, fast_score DESC);
CREATE INDEX IF NOT EXISTS idx_discovery_symbol_time
    ON discovery_candidates(symbol, discovered_at DESC);

CREATE TABLE IF NOT EXISTS funnel_snapshots (
    scan_id TEXT PRIMARY KEY,
    as_of TEXT NOT NULL,
    discovery_count INTEGER NOT NULL,
    stage0_survivors INTEGER NOT NULL,
    prescore_survivors INTEGER NOT NULL,
    options_prescreened INTEGER NOT NULL,
    final_candidates INTEGER NOT NULL,
    councils_started INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS execution_calibrations (
    calibration_id TEXT PRIMARY KEY,
    decision_id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    side TEXT NOT NULL,
    candidate_track TEXT NOT NULL,
    submitted_at TEXT NOT NULL,
    filled_at TEXT,
    indicative_raw_mid REAL NOT NULL,
    indicative_adjusted_mid REAL NOT NULL,
    natural_debit_estimate REAL NOT NULL,
```

```
    initial_limit_debit REAL NOT NULL,
    final_submitted_limit REAL NOT NULL,
    actual_fill_debit REAL,
    seconds_to_fill REAL,
    limit_walk_steps INTEGER NOT NULL DEFAULT 0,
    quote_lag_seconds REAL NOT NULL,
    underlying_at_quote REAL NOT NULL,
    underlying_at_submit REAL NOT NULL,
    underlying_at_fill REAL,
    fill_bias_vs_adjusted REAL,
    fill_bias_vs_limit REAL,
    fill_slippage_pct REAL
);
CREATE INDEX IF NOT EXISTS idx_exec_cal_symbol_time
    ON execution_calibrations(symbol, submitted_at DESC);
```

These are additive tables. Existing v2.3 databases SHOULD be migrated in place.

8.4 Altered tables

In addition to the v2.3 alterations, v2.4 adds source/track provenance:

```
ALTER TABLE candidate_scores ADD COLUMN discovery_source TEXT;
ALTER TABLE candidate_scores ADD COLUMN candidate_track TEXT;
ALTER TABLE decisions ADD COLUMN discovery_source TEXT;
ALTER TABLE decisions ADD COLUMN candidate_track TEXT;
```

If a column already exists, the migration runner MUST treat that as already-applied rather than failing.

Existing v2.3 alterations continue below:

```
ALTER TABLE decisions         ADD COLUMN config_version TEXT;
ALTER TABLE candidate_scores   ADD COLUMN config_version TEXT;
ALTER TABLE risk_evaluations   ADD COLUMN config_version TEXT;
ALTER TABLE option_structures  ADD COLUMN raw_mid_debit REAL;
ALTER TABLE option_structures  ADD COLUMN adjusted_mid_debit REAL;
ALTER TABLE option_structures  ADD COLUMN max_quote_lag_seconds REAL;
ALTER TABLE option_structures  ADD COLUMN net_delta REAL;
ALTER TABLE option_structures  ADD COLUMN cost_to_width_ratio REAL;
ALTER TABLE shadow_marks       ADD COLUMN mark_method TEXT;
ALTER TABLE shadow_marks       ADD COLUMN quote_lag_seconds REAL;
ALTER TABLE decision_attribution ADD COLUMN claude_selection_effect REAL;
ALTER TABLE decision_attribution ADD COLUMN claude_sizing_effect REAL;
ALTER TABLE decision_attribution ADD COLUMN risk_selection_effect REAL;
ALTER TABLE decision_attribution ADD COLUMN risk_sizing_effect REAL;
```

Each pipeline stage MUST persist its output before invoking the next stage.

---

9. Alpaca Integration

9.1 MCP configuration

```
{
  "servers": {
    "alpaca": {
```

```
    "command": "uvx",
    "args": ["alpaca-mcp-server"],
    "env": {
     "ALPACA_API_KEY": "${ALPACA_API_KEY}",
     "ALPACA_SECRET_KEY": "${ALPACA_SECRET_KEY}",
     "ALPACA_PAPER_TRADE": "true",
     "ALPACA_TOOLSETS": "account,trading,assets,stock-data,options-data,news"
    }
   }
  }
}
```

Required tools: get_account_info, get_clock, get_all_positions, get_open_position, get_option_contracts, get_option_snapshot, get_option_latest_quote, get_stock_latest_quote, get_stock_snapshot, get_news, place_option_order, get_order_by_id, get_order_by_client_id, get_orders, cancel_order_by_id, get_account_activities.

At startup call session.list_tools(), persist every tool name and JSON schema to system_events, and **fail startup if any execution-critical tool is missing**.

9.2 REST endpoints (batch work)

Core batch endpoints:

GET https://data.alpaca.markets/v2/stocks/snapshots?symbols=...&feed=iex
GET https://data.alpaca.markets/v2/stocks/bars?symbols=...&timeframe=5Min&start=...&end=...&feed=iex
GET https://data.alpaca.markets/v2/stocks/quotes/latest?symbols=...&feed=iex
GET https://data.alpaca.markets/v1beta1/news?start=...&end=...&symbols=...&limit=50&include_content=true
GET https://data.alpaca.markets/v1beta1/options/snapshots/{underlying}?feed=indicative&limit=1000
GET https://data.alpaca.markets/v1beta1/options/snapshots?symbols=OCC1,OCC2&feed=indicative

Optional discovery endpoints (probe entitlement once per session):

GET https://data.alpaca.markets/v1beta1/screener/stocks/most-actives?by=volume&top=100
GET https://data.alpaca.markets/v1beta1/screener/stocks/movers?top=50

The screeners are based on real-time SIP data and may be unavailable to the Basic competition account. Behavior:

- HTTP 200: normalize returned symbols into the Dynamic Discovery Universe.
- HTTP 403: log once, disable that discovery source for the session, continue normally.
- HTTP 429/5xx: normal retry/backoff policy.
- Screeners MUST NOT become execution dependencies.

Asset eligibility uses the Trading API assets endpoint and SHOULD filter for status=active, tradable US equities/ETFs, and has_options where supported.

Headers: APCA-API-KEY-ID, APCA-API-SECRET-KEY. Follow pagination tokens until absent. The REST client remains rate-limited below the account ceiling.

9.3 Order payload

```
{
 "order_class": "mleg",
 "qty": "1",
 "type": "limit",
 "limit_price": "2.35",
```

```
  "time_in_force": "day",
  "client_order_id": "ac_<8hex>_r<0|1>_<8hex>",
  "legs": [
   {"symbol": "<LONG_OCC>",  "ratio_qty": "1", "side": "buy",  "position_intent": "buy_to_open"},
   {"symbol": "<SHORT_OCC>", "ratio_qty": "1", "side": "sell", "position_intent": "sell_to_open"}
  ]
}
```

A positive mleg limit price is a debit. Closing orders reverse to sell_to_close / buy_to_close.

---

10. Intelligence Plane - Focused OSINT Plus Discovery Injection

Primary collectors remain **SEC EDGAR** and **Alpaca News**. v2.4 adds one requirement: intelligence may temporarily inject an otherwise off-core symbol into discovery; intelligence never bypasses the normal quantitative/options gates.

10.1 SEC EDGAR

For Core and currently active dynamic symbols:

User-Agent: AlphaCouncil/0.1 operator@example.com
Accept-Encoding: gzip, deflate

GET https://www.sec.gov/files/company_tickers.json
authenticated/no-key SEC submissions JSON for mapped CIKs

Read recent accession number, accepted timestamp, form, primary document, and items. Retrieve the primary document only for priority forms: **8-K, 10-Q, 10-K, 6-K, Form 4, 13D/13G, S-3, 424B***.

Off-core discovery (SHOULD, not MVP-blocking): if a stable SEC current-filings/Atom/RSS feed is implemented and verified against official SEC behavior, map newly observed priority filings to ticker via the CIK map. If the mapped asset is Alpaca-tradable and option-enabled, add it to EventInjectedUniverse for the configured TTL. If this global feed is unavailable or changes, log degradation and continue active-universe SEC polling; do not build a brittle scraper.

10.2 Alpaca News

Poll Alpaca News every 60 seconds during RTH. In addition to symbol-filtered news for the active set, maintain a lightweight market-wide recent-news query when the endpoint permits it. For every tagged symbol not currently in Core:

1. resolve the Alpaca asset;
2. require active/tradable status and options availability;
3. add to EventInjectedUniverse with source ALPACA_NEWS and TTL (default 90 minutes);
4. run the normal Stage-0 fast screen before any options or LLM work.

No headline can directly generate an order.

10.3 Scoring

Source reliability, freshness half-life, novelty, corroboration clustering, materiality, surprise, and market confirmation carry over from v2.3.

CatalystScore = 0.30*Materiality + 0.20*Freshness + 0.20*SourceReliability
+ 0.15*MarketConfirmation + 0.15*Surprise

MarketConfirmationScore uses IEX-derived returns and same-feed RVOL. Macro releases remain blackout/regime inputs, not single-name catalysts.

10.4 Candidate tracks

Every candidate is labeled before Council:

EVENT track - material fresh SEC/news evidence exists; - market response is directionally confirmatory or meaningfully rejects the news; - catalyst, novelty, and corroboration features are active.

MOMENTUM track - no material catalyst is required; - price momentum, RVOL, relative strength, trend/regime, and options quality must be unusually strong; - known intelligence must not materially contradict the direction.

Both tracks converge on the same options engine, GPT PM, Claude Red Team, Risk Constitution, execution and attribution. The Momentum track prevents Alpha Council from missing a market move merely because the news pipeline has not yet explained it.

11. Single-Source Data Quality Guard

Replaces the v2.2 two-provider consensus engine.

11.1 Equity (IEX, real-time)

mid = (bid + ask)/2 if valid, else last, else close
spread_pct = (ask - bid) / max(mid, 0.01)

Condition	Confidence	Factor
lag ≤ 5s and spread_pct ≤ 0.005	HIGH	1.00
lag ≤ 30s and spread_pct ≤ 0.015	MEDIUM	0.92
lag ≤ 120s or spread_pct ≤ 0.030	DEGRADED	0.80
lag > 120s, or ask < bid, or negative price	BLOCKED	0.00

Additional sanity checks that replace a second provider:

1. **Cross-check within Alpaca:** latest quote mid vs latest trade price vs latest minute-bar close. If the maximum pairwise divergence exceeds 1.5%, downgrade one level and log DQ_INTERNAL_DIVERGENCE.
2. **Bar continuity:** if the last 5-minute bar is more than 10 minutes old during RTH, downgrade to DEGRADED.
3. **Halt detection:** if no trades for 5 minutes on a normally liquid name, treat as BLOCKED.

execution_reference_price MUST always be the current Alpaca IEX midpoint.

11.2 Options (Indicative Pricing Feed)

Evaluate **timestamp freshness and quote plausibility separately**. A quote can be timestamp-fresh and still be indicative rather than OPRA NBBO.

Condition	Confidence	Factor
lag <= measured fresh threshold, valid bid/ask, drift <= 0.25%	HIGH	1.00
lag <= 300s and drift <= 0.5%	MEDIUM	0.92
lag <= configured max and drift <= 1.0%	DEGRADED	0.80
lag > configured max, drift > 1.0%, bid <= 0, ask <= 0, ask < bid, or missing Greeks	BLOCKED	0.00

Rules:

1. ask <= 0, bid <= 0, or ask < bid is invalid and MUST be rejected before computing a midpoint.
2. Apply §5.4 only when timestamp freshness requires it.
3. Regardless of timestamp freshness, use conservative limit pricing and record execution calibration because Indicative != OPRA NBBO.
4. Any structure whose worst leg is BLOCKED is rejected with OPT_QUOTE_BLOCKED.
5. If observed Monday quote timestamps are consistently near-real-time, tighten lag thresholds rather than disabling the indicative-quality safeguards.

---

12. Market Observatory, Discovery, Scoring, and Breadth-First Adaptation

12.1 Universe manager and Stage-0 discovery

Maintain four sets:

```
CoreUniverse            # static ~65
ScreenerInjectedUniverse # most-active / movers; TTL
EventInjectedUniverse    # news / SEC; TTL
DynamicDiscoveryUniverse # union, de-duped, capped 250
```

The cap is enforced deterministically. When >250 eligible symbols exist, retain in this order:

1. Core symbols.
2. Fresh EVENT-injected symbols by materiality/freshness.

3. Most-active/mover symbols by discovery rank.
4. Remaining eligible symbols by IEX data density and recent dollar-volume proxy.

Stage-0 uses only batch equity data and cached asset metadata. It MUST NOT fetch option chains or invoke LLMs.

Stage-0 FastScore (0-100):

```
FastScore = 0.30*abs(MomentumScore - 50)*2
          + 0.25*RelativeVolumeScore
          + 0.20*RelativeStrengthScore
          + 0.15*TrendRegimeScore
          + 0.10*DiscoveryBoost
```

Where DiscoveryBoost = 100 for a fresh material EVENT injection, 80 for top-10 mover/most-active, 50 for other dynamic discovery, 40 for Core-only. Clip all components to 0-100. Select top 30 after hard data-quality and direction-ambiguity filters.

12.2 Bar history

Load 20 trading days of **regular-trading-hours-only** 5-minute IEX bars for the active discovery set. Do not include extended-hours bars in the RVOL baseline. Cache history and backfill newly injected symbols on demand.

12.3 Direction signal

```
mom_signed   = tanh((0.40*r5 + 0.35*r15 + 0.25*r60) / 0.01)
rs_signed    = tanh((0.50*rs15 + 0.50*rs60) / 0.01)
trend_signed = mean([sign(price - VWAP), sign(EMA9 - EMA20),
                     sign(price - day_open), benchmark_alignment])
tech_direction = 0.50*mom_signed + 0.30*rs_signed + 0.20*trend_signed

combined_direction = 0.65*tech_direction + 0.35*catalyst_direction  # EVENT track
combined_direction = tech_direction                                 # MOMENTUM track
```

abs(combined_direction) < 0.15 -> reject DIR_AMBIGUOUS.

12.4 Relative volume (IEX-adjusted)

```
RVOL = iex_volume_current_15m_window /
       median(iex_volume_same_clock_window over prior 20 RTH sessions)
RelativeVolumeScore = clip(40 + 30*log2(max(RVOL, 0.25)), 0, 100)
```

Never compare IEX volume to a published consolidated-volume figure.

12.5 PreScore and track-aware scoring

Base PreScore:

```
PreScore = 0.20*Momentum + 0.20*RelativeVolume + 0.15*TrendRegime
         + 0.15*RelativeStrength + 0.20*Catalyst
         + 0.05*Corroboration + 0.05*Novelty
```

For **MOMENTUM** candidates with no material catalyst, do not inject an artificial bullish/bearish catalyst. Reallocate the 30% catalyst/corroboration/novelty weight proportionally across Momentum, RVOL, Trend/Regime, and Relative Strength:

```
MomentumPreScore = 0.30*Momentum + 0.30*RelativeVolume
                 + 0.20*TrendRegime + 0.20*RelativeStrength
```

Rank the top 12 Stage-1 candidates for options pre-screen.

12.6 Final Opportunity Score

EVENT track:

```
RawOpportunityScore = 0.15*Momentum + 0.15*RelativeVolume + 0.10*TrendRegime
                    + 0.10*RelativeStrength + 0.10*OptionsOpportunity
```

+ 0.10*OptionsLiquidity + 0.20*Catalyst
+ 0.05*Corroboration + 0.05*Novelty

MOMENTUM track:

RawOpportunityScore = 0.22*Momentum + 0.22*RelativeVolume + 0.14*TrendRegime
+ 0.14*RelativeStrength + 0.14*OptionsOpportunity
+ 0.14*OptionsLiquidity

For both:

FinalOpportunityScore = RawOpportunityScore * DataConfidenceFactor
* RegimeFactor * EventRiskFactor

Return top 5 final candidates; normal scans may start up to 3 Councils.

12.7 Breadth-First Adaptive Ladder

The anti-zero-trade mechanism MUST expand breadth before degrading trade quality.

Session sequence:

Time/condition	Action
Session start	Tier 1 quality + Core + normal dynamic discovery
11:00 ET and zero orders	expand discovery sources/cap to full 250; refresh movers/most-active/news injections
12:30 ET and zero orders	Tier 2 quality gates
14:00 ET and zero orders	second breadth refresh; promote fresh event/momentum outliers
14:15 ET and zero orders	Tier 3 score/confidence relaxation only; preserve meaningful liquidity
Any order submitted	hold current tier; no ratchet back
Competition genuine-alpha orders >= 14	pin Tier 1 for remainder

Quality gates:

Gate	Tier 1	Tier 2	Tier 3
PreScore floor	62	56	52
FinalOpportunityScore floor	68	62	58
PM confidence floor	0.60	0.55	0.52
Max cost/width ratio	0.55	0.60	0.62
Long leg abs(delta)	0.52-0.72	0.48-0.78	0.46-0.80
Short leg abs(delta)	0.22-0.42	0.18-0.46	0.17-0.48
Min leg open interest	250	100	75
Min leg session volume	25	10	**5**
Max leg spread_pct	0.15	0.20	**0.22**
DTE window	7-21	5-30	5-30
Max councils per scan	3	3	4
Max councils per day	12	16	18

Hard rule: Tier 3 MUST NOT set minimum option volume to zero and MUST NOT permit ~28% bid/ask spreads merely to create activity. If no valid structure exists after breadth expansion, NO TRADE is acceptable.

Reward/risk remains derived from cost/width:

RR = (1 - c/w) / (c/w)

12.8 Calibration

Run scripts/gate_report.py --dry in log-only mode. Report:

- elimination counts by gate;
- elimination counts by universe source (Core, Event, Movers, MostActive);
- candidate survival at each funnel stage (Discovery -> 30 -> 12 -> 5 -> Council);
- track split (EVENT vs MOMENTUM).

If one non-safety gate eliminates >60% of otherwise-valid structures, review the observed distribution before live trading. Do not alter a liquidity floor solely to satisfy an activity quota.

13. Deterministic Options Engine

13.1 Chain acquisition

For each eligible symbol, request the chain for the tier's DTE window via `/v1beta1/options/snapshots/{underlying}?feed=indicative`. Cache chains for 60 seconds per underlying to stay inside the rate limit.

Per-leg hard filters (tier values from §12.6):

DTE within tier window and DTE >= 3 and not 0DTE
bid > 0 and ask > bid
Greeks and IV present
open_interest >= tier.min_open_interest
volume >= tier.min_volume
spread_pct <= tier.max_leg_spread_pct
quote lag and underlying drift within §11.2 bounds

Both legs MUST share underlying, expiration, and option type.

13.2 Pricing with Indicative quotes

```
adjusted_long_mid  = long_raw_mid  + long_delta  * underlying_move
adjusted_short_mid = short_raw_mid + short_delta * underlying_move

mid_debit     = adjusted_long_mid - adjusted_short_mid
natural_debit = long_ask - short_bid          # raw, unadjusted, conservative
staleness_buffer = max(0.02, 0.05 * mid_debit) if max_leg_lag > fresh_quote_seconds else 0.0
```

Reject if `mid_debit <= 0`, `natural_debit <= 0`, or `natural_debit >= width`.

13.3 Payoff math

```
width              = abs(short_strike - long_strike)
cost_to_width_ratio  = limit_debit / width
max_loss_per_spread  = limit_debit * 100
max_profit_per_spread= (width - limit_debit) * 100
reward_risk_ratio    = max_profit_per_spread / max_loss_per_spread
breakeven          = long_strike + limit_debit   (bull call)
                   = long_strike - limit_debit   (bear put)
```

All sizing and risk math MUST use the **limit debit actually submitted**, never the mid.

Hard rejects: `cost_to_width_ratio > tier.max_cost_to_width`, `max_profit_per_spread <= 0`, `width <= 0`.

13.4 Initial limit debit

```
initial_limit_debit = mid_debit + 0.25*(natural_debit - mid_debit) + staleness_buffer
initial_limit_debit = min(initial_limit_debit, natural_debit, max_allowed_debit_from_risk)
```

Round to the option tick. The staleness/indicative buffer is the price of uncertainty in the free Indicative feed; it is recorded separately. Once §17.5 has at least three calibration fills, the engine MAY add a bounded empirical `fill_bias_buffer` derived from recent same-direction debit-spread fills. The empirical buffer MUST be capped and MUST NOT override `natural_debit` or risk limits.

13.5 Scoring

```
SpreadScore  = 100 * clip(1 - spread_pct / tier.max_leg_spread_pct, 0, 1)
OIScore      = 100 * clip(log1p(open_interest) / log1p(5000), 0, 1)
VolumeScore  = 100 * clip(log1p(volume) / log1p(2000), 0, 1)
FreshScore   = 100 if lag<=60s; 80 if <=300s; 60 if <=900s; 30 if <=1200s; else 0
```

```
LegLiquidityScore = 0.45*SpreadScore + 0.30*OIScore + 0.15*VolumeScore + 0.10*FreshScore
LiquidityScore    = min(leg scores)

RRScore   = clip(60 * reward_risk_ratio, 0, 100)
DTEFit    = max(60, 100 - 5*abs(DTE - 14))
DeltaFit  = mean(100*clip(1-abs(abs(long_delta)-0.60)/0.20, 0, 1),
             100*clip(1-abs(abs(short_delta)-0.33)/0.18, 0, 1))
CostEff   = 100 * clip(1 - cost_to_width_ratio, 0, 1)

StructureScore = 0.30*LiquidityScore + 0.25*RRScore + 0.20*DeltaFit
          + 0.15*DTEFit + 0.10*CostEff
```

Return the **top five** structures by score, requiring at least two distinct strike pairs. Set OptionsOpportunityScore = best StructureScore and OptionsLiquidityScore = best structure LiquidityScore.

Every structure eliminated at this stage writes a gate_rejections row with stage = OPTIONS_STRUCTURE.

---

14. AI Model Allocation and Budget

14.1 Cost model

```
cost = input_tokens/1e6 * input_price + output_tokens/1e6 * output_price
```

Reasoning/thinking tokens bill as output on all three models. Prices live in config/scoring.yaml, not in code, and MUST be verified against each provider's console before the first live session.

Model	Input $/MTok	Output $/MTok
GPT-5.6 Luna	0.20	1.20
GPT-5.6 Sol	4.00	20.00
Claude Sonnet 5	2.00	10.00

14.2 Assignment

Task	Model	Setting	Evidence cap
Bull analyst	gpt-5.6-luna	reasoning low	3,500 tok
Bear analyst	gpt-5.6-luna	reasoning low	3,500 tok
Catalyst analyst	gpt-5.6-luna	reasoning medium	3,500 tok
Portfolio manager	gpt-5.6-sol	reasoning medium	6,000 tok
Structure selection	gpt-5.6-sol	reasoning medium	3,000 tok
PM revision	gpt-5.6-sol	reasoning medium	7,000 tok
Red Team	**claude-sonnet-5**	**effort high**	8,000 tok
Post-close lessons	claude-sonnet-5	effort medium	6,000 tok

14.3 Budget projection

Per council session, with a 40% probability of a revision round:

Call	Model	In	Out	Cost
Bull	Luna	3.5k	0.7k	$0.0016
Bear	Luna	3.5k	0.7k	$0.0016
Catalyst	Luna	3.5k	0.9k	$0.0018
PM propose	Sol	6.0k	1.2k	$0.048
Structure select	Sol	3.0k	0.5k	$0.022
PM revision (×0.4)	Sol	7.0k	1.2k	$0.021
OpenAI per session				**≈ $0.096**
Red Team	Sonnet 5	8.0k	2.0k	$0.036
Anthropic per session				**≈ $0.036**

At the Tier-1 cap of 12 councils/day across 5 sessions (60 sessions):

	Projected	3× safety factor	Budget	Headroom
OpenAI	$5.76	$17.28	$50	65% unused
Anthropic	$2.16	$6.48	$50	87% unused

Finding: the $50 caps are not the binding constraint. Underuse is. Even at Tier 3 (20 councils/day) with triple the estimated reasoning tokens, spend lands near $29 / $11.

Therefore:

1. Keep Sol for the PM. It is called 2-3 times per session and is the highest-leverage decision in the pipeline. Downgrading it to save $12 would be a false economy.
2. Keep Sonnet 5 at effort=high for the Red Team. This is the model whose objections you are trying to measure; do not starve it.
3. Spend surplus on quality, in this order:
   - Raise the Red Team evidence cap to 12,000 tokens once measured spend confirms headroom (after Day 1).
   - Add a **pre-market briefing call** (Sol, 08:45 ET) summarizing overnight SEC filings and news into a session context block reused by every council that day. Prompt-cacheable, roughly $0.05/day.
   - Add a **post-close lessons call** (Sonnet 5, 16:15 ET) that reads the day's decisions, fills, and gate rejections and writes trade_journal.lesson plus a calibration recommendation. Roughly $0.06/day. This is also a strong dashboard artifact.
4. Ordering discipline: stable prompt text first, dynamic evidence last, to maximize prompt-cache reuse.

14.4 Enforcement

```
per_session_cost_ceiling: $0.75     # abort session, log BUDGET_SESSION_CAP
daily_provider_ceiling:   $8.00     # block new councils for that provider
openai:   < $40 normal | $40-48 reserve (PM + Red Team only) | >= $48 block
anthropic:< $40 normal | $40-48 reserve (Red Team only)      | >= $48 block
```

Every call writes to api_usage with decision_id, token counts, and cost. **Reconcile measured spend against this projection at the end of Day 1 and adjust caps once.**

---

15. Agent Prompts

The six v2.3 prompt files carry over with these v2.4 amendments:

All prompts gain:

```
The option prices in this evidence package come from Alpaca's Indicative Pricing Feed,
not OPRA NBBO. A quote timestamp may be fresh while the quoted value is still an
indicative/derived estimate. Each option field carries quote_lag_seconds, raw_mid,
adjusted_mid, and stale_adjusted. When stale_adjusted is true, the adjusted price uses
underlying movement since the quote timestamp. Treat all indicative prices as estimates.
Do not infer intraday option-price momentum from indicative trade prints.
```

red_team_system.txt gains a fifth audit layer:

```
5. DATA FIDELITY / STALENESS - Given the reported quote timestamp, quote_lag_seconds,
   underlying drift, Indicative-feed limitations, and execution-calibration history, is
   the proposed debit plausible? Is the thesis dependent on pricing precision the feed
   cannot support? Would a modest indicated-to-fill bias invalidate the reward/risk?
```

pm_system.txt gains:

```
Invalidation conditions MUST be expressible in terms of the UNDERLYING price, VWAP, or
elapsed time. Do not write invalidation rules that depend on the option's own price,
because the system cannot observe option prices in real time.
```

Structure-selection prompt: selected_structure_rank range becomes 1-5.

Evidence Package contract is unchanged except top_option_structures now carries up to five entries and each leg includes quote_lag_seconds, raw_mid, adjusted_mid.

---

16. Risk Constitution

16.1 config/risk_constitution.yaml

```yaml
config_version: v1
paper_only: true

# ---- HARD GATES: never relaxed by the adaptive ladder ----
hard:
  max_risk_per_trade_pct: 2.0
  max_total_open_option_risk_pct: 10.0
  max_sector_open_risk_pct: 4.0
  max_concurrent_positions: 5
  max_daily_drawdown_pct: 5.0
  max_competition_peak_drawdown_pct: 12.0
  min_dte: 3
  no_0dte: true
  allow_naked_options: false
  allow_credit_spreads: false
  allow_average_down: false
  defined_risk_only: true
  two_legs_only: true
  new_trade_cutoff_et: "15:20"
  force_close_before_expiration_days: 2
  competition_flatten_et: "2026-09-03T15:45:00-04:00"

# ---- QUALITY GATES: tier-driven, see scoring.yaml ----
quality:
  target_risk_per_trade_pct: 1.25
```

16.2 Hard check order

Evaluate in sequence, collecting all violations:

1. Alpaca paper-mode assertion → **HALT** on failure.
2. Market open and before new_trade_cutoff_et.
3. Data quality not BLOCKED (equity and both option legs).
4. Claude verdict != VETO.
5. Event blackout clear.
6. Strategy is an allowed two-leg defined-risk debit vertical.
7. DTE ≥ 3 and not 0DTE.
8. Reward/risk ≥ tier floor and cost/width ≤ tier ceiling.
9. Per-trade max loss ≤ 2% equity.
10. Total open option risk after trade ≤ 10% equity.
11. Sector open risk after trade ≤ 4% equity.
12. Concurrent positions after trade ≤ 5.
13. Daily drawdown < 5% → **HALT** on failure.
14. Competition peak-to-trough drawdown < 12% → **HALT** on failure.
15. No duplicate or open order for the same decision_id.

Failure of 1, 13, or 14 → HALT. Any other hard failure → REJECT. Quantity above allowed risk only → RESIZE. Every violation writes a gate_rejections row with hard_gate = 1.

16.3 Position sizing

```
requested_risk = equity * desired_portfolio_risk_pct / 100
hard_cap       = equity * 0.02
red_team_cap   = equity * recommended_max_risk_pct / 100
risk_budget    = min(requested_risk, hard_cap, red_team_cap)
qty            = floor(risk_budget / max_loss_per_spread)
```

qty < 1 → reject with gate RISK_QTY_ZERO. Record requested_qty and approved_qty separately; the attribution decomposition depends on both.

16.4 Event blackout

```yaml
events:
  - name: US International Trade in Goods and Services
    source: BEA
    timestamp_et: "2026-09-03T08:30:00-04:00"
    pre_block_minutes: 15
    post_block_minutes: 5
  - name: Employment Situation - August 2026
    source: BLS
    timestamp_et: "2026-09-04T08:30:00-04:00"
    pre_block_minutes: 15
    post_block_minutes: 5
```

Add verified earnings dates for active-universe symbols as they are confirmed. Do not infer a schedule from model memory. Default earnings rule: block initiating a directional spread from T-30m through the first 10 minutes of the following session.

EventRiskFactor remains binary: 1.00 allowed, 0.00 blocked.

---

17. Execution and Idempotency

17.1 Decision state machine

```
CANDIDATE -> COUNCIL_STARTED -> PM_PROPOSED -> STRUCTURES_GENERATED
      -> STRUCTURE_SELECTED -> RED_TEAMED -> [REVISED]
      -> RISK_APPROVED | RISK_REJECTED
      -> ORDER_SUBMITTED -> ORDER_WORKING
      -> FILLED | CANCELED | REJECTED | NO_FILL
      -> POSITION_OPEN -> POSITION_CLOSED -> ATTRIBUTED
```

Every transition is persisted.

17.2 Idempotency

client_order_id = ac_<8hex>_r<0|1>_<8hex>. Before any retry: call get_order_by_client_id; if an order exists, adopt it and do not resubmit. Retry only when the original call demonstrably failed before acceptance.

17.3 Limit walk (conservative for Indicative quotes)

```
Attempt 1: mid + 25% of (natural - mid) + staleness_buffer   ... wait 30s
Attempt 2: mid + 60% of (natural - mid) + staleness_buffer   ... wait 30s
Attempt 3: min(natural, max_allowed_debit_from_risk)
```

Maximum three prices. Never exceed natural_debit or the Risk Constitution's dollar ceiling. If unfilled after attempt 3, cancel and record NO_FILL with a gate_rejections row (stage = EXECUTION, gate_id = EXEC_NO_FILL) so unfilled intent is still measurable.

17.4 Pre-submit refresh

Immediately before each submission or replacement: refresh the underlying quote (must be ≤ 5s old), refresh both option legs, recompute the stale-quote adjustment, recompute max-loss math against the proposed limit, and rerun the Risk Constitution. **No stale approval may be reused.**

17.5 Execution/Fill Calibration Engine

Every submitted opening and closing spread MUST create an execution-calibration record. Capture:

```
decision_id
underlying
submitted_at
indicative_raw_mid
indicative_adjusted_mid
natural_debit_estimate
initial_limit_debit
final_submitted_limit
actual_fill_debit
seconds_to_fill
limit_walk_steps
underlying_at_quote
underlying_at_submit
underlying_at_fill
quote_lag_seconds
track            # EVENT | MOMENTUM | CALIBRATION
```

Derived metrics:

```
fill_bias_vs_adjusted = actual_fill_debit - indicative_adjusted_mid
fill_bias_vs_limit    = actual_fill_debit - initial_limit_debit
fill_slippage_pct     = fill_bias_vs_adjusted / max(indicative_adjusted_mid, 0.01)
```

After at least three valid liquid-spread fills, compute rolling median and 80th-percentile fill bias. This is **measurement**, not machine learning. It may inform a small bounded `fill_bias_buffer` for subsequent initial limits.

Rules:

- Never learn from obviously stale/unusable quotes.
- Keep opening and closing calibration separate.
- Keep bullish/bearish structures separate if sample size permits.
- Maximum learned buffer MUST be configured (default min($0.10, 5% of adjusted mid)).
- Dashboard MUST show mean/median indicated-to-fill difference, time to fill, and limit-walk steps.
- Execution calibration does not alter the risk calculation after an order is approved; if a higher debit would exceed max risk, stop the limit walk.

This engine is higher priority than adding Schwab.

---

18. Position Monitoring and Exits

Poll every 2 minutes during RTH. Each poll: fetch positions and order state, refresh the **underlying** quote, refresh option legs, recompute adjusted spread mark, update actual and shadow books, evaluate exits.

18.1 Exit rules

Primary triggers are computed from the **underlying**, which is real-time:

```
UNDERLYING_TARGET      close when underlying reaches the strike-based profit zone
                       (default: underlying >= short_strike for bull calls,
                        <= short_strike for bear puts)
```

UNDERLYING_INVALIDATION close when the PM's stated underlying/VWAP invalidation rule fires
TIME_STOP close at 2 DTE
CUTOFF close all positions at 2026-09-03 15:45 ET (competition flatten)

Secondary triggers use the adjusted option mark and are advisory unless data quality is HIGH or MEDIUM:

PROFIT_TARGET adjusted spread mark >= 55% of max profit
PREMIUM_STOP adjusted spread mark <= 45% of entry debit

A catalyst-based invalidation MAY trigger a fresh Red Team review, but **exit logic MUST NOT require an LLM call.** Existing positions must remain manageable with both AI providers down.

18.2 Competition flatten

scripts/close_all.py runs automatically at 15:45 ET on September 3 and is also runnable manually. Rationale: realized P&L in the submission beats open marks, and the September 4 submission window closes at 11:00 ET before the market opens meaningfully.

---

19. Counterfactual Shadow Book

19.1 Variants

- GPT_ORIGINAL - created at the first PM proposal, using the PM's originally selected structure and requested risk sizing.
- CLAUDE_MODIFIED - created when the Red Team returns MODIFY and the PM revises. On VETO, quantity is zero.
- EXECUTED - created from actual fill data.

19.2 Marking

All variants MUST be marked with the identical method at each timestamp, recorded in shadow_marks.mark_method:

mark_method = "ADJUSTED_MID" : adjusted_long_mid - adjusted_short_mid
mark_method = "CONSERVATIVE" : long_bid - short_ask (raw, unadjusted)

Default ADJUSTED_MID. Record quote_lag_seconds on every mark.

19.3 Attribution

Use the exact decomposition from §7.5. Report all four effects on the dashboard:

Claude selection effect : did the red team pick a worse or better structure?
Claude sizing effect : did the red team just make it smaller?
Risk selection effect : did the risk engine change the expression?
Risk sizing effect : did position sizing help or hurt?

Negative values are expected and MUST NOT be suppressed. "Our red team cost us $180 in selection but saved $410 in sizing" is a more credible finding than a uniformly positive result.

---

20. Gate Rejection Log and the Calibration Loop

20.1 What gets logged

Every gate that stops a candidate writes exactly one gate_rejections row at the point of rejection. A candidate that fails five gates writes five rows only if all five are evaluated; short-circuiting stages MUST log the first failing gate and set note to indicate that evaluation stopped.

Required fields: stage, gate_id, observed value, threshold value, tier, config_version, hard/quality flag.

20.2 Shadow-marking rejected candidates

A rejection is shadow_eligible when a fully-formed, priced structure existed at the moment of rejection - that is, rejections at stages OPPORTUNITY_SCORE, PM_ABSTAIN, RED_TEAM, RISK, and EXECUTION. For these, persist the structure to rejected_shadows and mark it on the same schedule as real shadow trades, until the earlier of the PM's stated horizon or the competition flatten.

This produces the second half of the demo claim: not just "what did each layer do to the trades we took," but "what did each gate cost us on the trades we didn't."

20.3 Using the log to improve the system - recommended practice

You asked how the rejection log should feed back into the model. Three separate mechanisms, in increasing order of caution:

(a) In-competition threshold calibration (do this; it is cheap and safe). Run scripts/gate_report.py after each close. It produces a histogram of gate_id → rejection count, plus, for shadow-eligible rejections, the mean and median hypothetical P&L per spread of what was blocked. Define:

GateValue(g) = -1 * mean(hypothetical_pnl_per_spread of trades rejected solely by g)

A gate with strongly positive GateValue is earning its place. A gate with persistently negative GateValue is systematically blocking profitable trades and its Tier-1 value should move. Change **at most one quality gate per day**, write a new config_versions row, and note the reason. Never touch a hard gate this way.

(b) Prompt and evidence iteration (do this selectively). Where the PM abstains frequently or the Red Team vetoes on the same category repeatedly, the fix is usually a missing field in the evidence pack, not a threshold. gate_rejections grouped by RedTeamProblem.category tells you which. Change the evidence pack, not the prompt's opinions.

(c) Statistical fitting or fine-tuning (do NOT do this during the competition). With five sessions and on the order of tens of decisions, any weight optimization or model fine-tuning will fit noise. The sample is far too small for the results to generalize, and a judge who asks "how many observations is that tuned on?" will expose it. Say so explicitly in the README: the calibration loop is presented as *instrumentation that would enable* learning at scale, and the competition run is presented as its first, deliberately small sample. That framing is both honest and more impressive than a fake optimizer.

The correct post-competition path is: accumulate several hundred shadow-marked rejections across months, then fit gate thresholds by measured GateValue with proper out-of-sample splits. Build the plumbing now; run the fit later.

20.4 Dashboard surface

A "Gate Lab" tab showing: rejections by gate, tier timeline for each session, GateValue per gate with sample size prominently displayed, and a table of the ten most profitable blocked trades.

---

21. Scheduler and Orchestration

21.1 Schedule (America/New_York)

08:45 pre-market briefing call (Sol) + SEC/news overnight sweep
09:35 discovery refresh: Core + optional most-active/movers + event injections
09:40 full deterministic funnel scan
10:15 full deterministic funnel scan
11:00 breadth-expansion check if zero orders
11:30 full deterministic funnel scan
12:30 Tier-2 check if zero orders
13:30 full deterministic funnel scan
14:00 second breadth-expansion check if zero orders
14:15 Tier-3 check if zero orders
15:00 final full scan
15:20 new-trade cutoff

15:45 (Sep 3 only) competition flatten
16:15 post-close lessons + gate report + execution-calibration report

Every 5 minutes during RTH: refresh currently active discovery snapshots, poll Alpaca News, process new SEC items, expire TTL-based dynamic symbols, run position monitor, and event-trigger a PreScore for material intelligence or extreme momentum.

21.2 Universe orchestration

```
async def refresh_discovery_universe() -> list[str]:
    symbols = set(core_universe)
    symbols |= await event_injections.active_symbols()

    if discovery_sources.most_active_enabled:
        symbols |= await discovery_sources.try_most_active()
    if discovery_sources.movers_enabled:
        symbols |= await discovery_sources.try_movers()

    symbols = await asset_filter.keep_active_tradable_optionable(symbols)
    symbols = await data_density_filter.keep_usable_iex(symbols)
    return universe_ranker.cap(symbols, max_symbols=250)
```

A 403 from a screener disables that source for the current session and logs the event; it does not fail the scan.

21.3 Candidate orchestration

```
async def run_scan(trigger: str) -> None:
    discovery = await refresh_discovery_universe()
    fast = await scanner.stage0_fast_screen(discovery)
    top30 = fast[:30]

    pre = await scanner.full_prescore(top30)
    top12 = pre[:12]

    option_candidates = await options_engine.pre_screen(top12)
    final_candidates = scanner.final_rank(option_candidates)[:5]

    for candidate in final_candidates[:tier_manager.max_councils_per_scan()]:
        await evaluate_symbol(candidate.symbol, trigger=trigger)
```

evaluate_symbol() then follows the established Council -> PM -> options selection -> Red Team -> revision -> Risk -> Alpaca execution flow. Every candidate records universe_source and candidate_track.

21.4 Replay

There is no separate replay module. scripts/council_once.py --decision-id <id> re-runs the decision path against stored rows with injected clock/providers. Replays MUST preserve the original discovery source, candidate track, tier, and config version.

22. Build Order

Use this as a forward build plan from the current v2.3 status. Do not repeat already-completed work. Section 31 lists the patch sequence Claude Code/Codex should apply immediately.

Already completed from v2.3 - preserve

- environment/secrets handling and paper-only assertion;
- SQLite schema foundation and engine;
- Alpaca REST client, rate limiter and pagination;
- scoring/risk/universe configuration foundations;

- data-reality probe and Alpaca API smoke tests.

Phase A - Current weekend priority (Aug 29-30)

#	Deliverable
1	Pydantic domain models + repositories + utils
2	RTH-only market-data adapter/backfill; reject invalid bid/ask
3	quant/discovery.py: Core + Dynamic Discovery Universe, TTLs, source labels, cap 250
4	optional Alpaca most-active/movers adapters with 403 session-disable behavior
5	Stage-0 FastScore and funnel 250 -> 30 -> 12
6	quant indicators/regime/features/scoring + EVENT/MOMENTUM tracks
7	options engine and Indicative-feed quality logic
8	SEC + Alpaca News intelligence, including off-core event injection where practical
9	Bull/Bear/Catalyst, PM, Claude Red Team, budget manager
10	risk constitution + position sizing + blackout + gate logging

Checkpoint B/C: deterministic funnel produces ranked candidates; one stored candidate completes the full Council with validated structured outputs.

Phase B - Monday Aug 31: first supervised live session

1. Re-run probe_data_reality.py during RTH and lock quote-lag thresholds.
2. Run gate_report.py --dry across Core + dynamic discovery.
3. Verify movers/most-active entitlements; leave disabled on 403.
4. Complete orchestrator/scheduler with breadth-first adaptation.
5. Run dry trade; then one small supervised calibration/lifecycle spread.
6. Implement/verify execution/fill_calibration.py logging from the first fill.
7. Only after calibration, allow qualifying alpha entries.

Phase C - Sep 1: attribution and calibration

- trade journal, shadow book, rejected shadows, exact attribution decomposition;
- execution-calibration summaries;
- post-close lessons call;
- verify candidate-source and track attribution.

Phase D - Sep 2: dashboard and freeze

Dashboard tabs: Command Center, Discovery Funnel, Scanner, Council Decision, Counterfactual Lab, Gate Lab, Execution Quality, Audit. Freeze feature scope end of day.

Phase E - Sep 3: last trading day + submission assets

09:30-15:20 trade/monitor only, no refactors
15:45 automatic flatten
16:15 final gate + execution calibration report
Evening README, diagram, screenshots, video

Sep 4: submit

No trading and no new code. Submit before the competition deadline.

23. Acceptance Gates

Safety - [ ] dedicated competition paper account confirmed; Level 3 options verified - [ ] ALPACA_PAPER_TRADE=true; no live keys - [ ] risk tests pass including live-mode rejection and drawdown HALT - [ ] idempotency test passes - [ ] one spread opens/closes correctly

Data and Indicative-feed handling - [ ] Monday RTH probe recorded and lag config derived from measurement - [ ] invalid ask <= 0, bid <= 0, and ask < bid block midpoint use - [ ] stale-quote delta adjustment unit-tested - [ ] underlying-drift breach blocks structure - [ ] missing Greeks rejects structure - [ ] docs/README no longer claim every indicative quote is 15 minutes old

Discovery breadth - [ ] Core Universe always included - [ ] Dynamic Discovery Universe capped at 250 - [ ] Stage-0 produces top 30 without option-chain or LLM calls - [ ] top 12 alone receive options pre-screen - [ ] final top 5 and max councils are enforced - [ ] most-active/movers 403 disables only that source - [ ] fresh Alpaca News can inject eligible off-core symbol - [ ] each candidate records discovery source and TTL

Candidate tracks - [ ] EVENT and MOMENTUM scoring paths tested independently - [ ] MOMENTUM path does not fabricate neutral/fake catalyst direction - [ ] both tracks converge on same Council/Red Team/Risk pipeline

Intelligence - [ ] exact duplicate within source ignored - [ ] independent corroboration clusters correctly - [ ] freshness decay works - [ ] intelligence cannot bypass quant/options gates

AI - [ ] every model output validates through schema - [ ] PM can abstain - [ ] Claude can PASS/MODIFY/VETO - [ ] Claude VETO cannot be bypassed - [ ] provider outage blocks new councils but not position monitoring

Breadth-first adaptive ladder - [ ] 11:00 breadth expansion occurs before Tier 2 - [ ] Tier 2 only if zero orders at 12:30 - [ ] second breadth refresh before Tier 3 - [ ] Tier 3 minimum option volume >= 5 and max spread <= 22% - [ ] hard gates never relax

Execution calibration - [ ] every filled spread logs indicated/adjusted reference, submitted limit, fill, time and limit-walk steps - [ ] fill-bias metrics reconcile - [ ] any learned buffer is bounded and cannot exceed risk/natural-debit constraints

Attribution / demo - [ ] GPT original, Claude modified and executed variants mark side-by-side - [ ] rejected candidates can be shadow-marked where feasible - [ ] selection/sizing decomposition reconciles exactly - [ ] at least six order lifecycles demonstrated without requiring six qualifying alpha trades - [ ] dashboard explains one decision and its discovery path end-to-end

24. Dashboard

Use Streamlit. No Next.js before submission.

Required tabs:

1. **Command Center** - equity, P&L, open risk, positions, current tier, kill-switch state.
2. **Discovery Funnel** - Core size, dynamic size, source counts, 250 -> 30 -> 12 -> 5 -> councils; EVENT vs MOMENTUM split.
3. **Scanner** - candidate table with FastScore, PreScore, FinalOpportunityScore, source and track.
4. **Council Decision** - Bull/Bear/Catalyst, PM proposal, real option structures, Claude verdict, revision, risk result.
5. **Counterfactual Lab** - GPT-original vs Claude-modified vs executed P&L; selection and sizing effects.
6. **Gate Lab** - rejections by gate, tier timeline, GateValue with sample size.
7. **Execution Quality** - indicative raw/adjusted reference vs actual fills, mean/median fill bias, seconds to fill, limit-walk steps.
8. **Audit** - configuration version, discovery source, input timestamps, order state transitions, API usage.

The judge should be able to answer three questions without narration:

- **Why did Alpha Council notice this symbol?**
- **Why did it take or reject the trade?**
- **Did the Red Team/risk/execution layers add or destroy value?**

25. Failure Handling

Failure	Behavior
Alpaca MCP unavailable	HALT execution; continue collection/analysis
Equity data stale/invalid	BLOCK new trades; continue position management conservatively
Indicative option quote beyond configured lag	BLOCK new structure
Indicative quote timestamp fresh but bid/ask invalid	BLOCK new structure
Underlying drift > threshold	reject OPT_STALE_DRIFT
Most-active/movers returns 403	disable only that discovery source for session
Dynamic discovery > 250	deterministic cap/ranking; never overflow into extra LLM calls
SEC unavailable	continue; log collector degradation
Global SEC discovery unavailable	active-universe SEC polling continues
OpenAI unavailable	no new Council; monitor continues
Anthropic unavailable	no new Council; Red Team remains mandatory
Both LLM providers down	existing positions exit on deterministic rules
SQLite locked	retry briefly; do not advance state on persistence failure
LLM malformed output/refusal	no trade + rejection row
Order response timeout	query by client_order_id before retry
Execution calibration missing	trade may proceed under conservative static limits; log degradation
Optional Schwab validator unavailable	ignore warning layer; Alpaca pipeline unchanged

Failure	Behavior
Drawdown kill switch	cancel working entries, block new entries, continue exits

26. Configuration

config/scoring.yaml - v2.4 normative additions/changes

```yaml
config_version: v2.4

discovery:
  core_universe_always: true
  max_dynamic_symbols: 250
  stage0_top_n: 30
  options_prescreen_top_n: 12
  final_candidate_top_n: 5
  dynamic_ttl_minutes: 90
  enable_most_active: true
  enable_movers: true
  disable_source_on_403_for_session: true
  event_injection_enabled: true

breadth_expansion:
  first_expand_et: "11:00"
  tier2_after_et: "12:30"
  second_expand_et: "14:00"
  tier3_after_et: "14:15"
  pin_tier1_after_alpha_orders: 14

tiers:
  1:
    pre_score_floor: 62.0
    final_score_floor: 68.0
    pm_confidence_floor: 0.60
    max_cost_to_width: 0.55
    long_delta: [0.52, 0.72]
    short_delta: [0.22, 0.42]
    min_open_interest: 250
    min_volume: 25
    max_leg_spread_pct: 0.15
    dte: [7, 21]
    max_councils_per_scan: 3
    max_councils_per_day: 12
  2:
    pre_score_floor: 56.0
    final_score_floor: 62.0
    pm_confidence_floor: 0.55
    max_cost_to_width: 0.60
    long_delta: [0.48, 0.78]
    short_delta: [0.18, 0.46]
    min_open_interest: 100
```

```
    min_volume: 10
    max_leg_spread_pct: 0.20
    dte: [5, 30]
    max_councils_per_scan: 3
    max_councils_per_day: 16
  3:
    pre_score_floor: 52.0
    final_score_floor: 58.0
    pm_confidence_floor: 0.52
    max_cost_to_width: 0.62
    long_delta: [0.46, 0.80]
    short_delta: [0.17, 0.48]
    min_open_interest: 75
    min_volume: 5
    max_leg_spread_pct: 0.22
    dte: [5, 30]
    max_councils_per_scan: 4
    max_councils_per_day: 18

options:
  fresh_quote_seconds: 60       # replace after Monday RTH probe
  max_quote_lag_seconds: 1200   # replace after Monday RTH probe
  max_underlying_drift_pct: 0.010
  chain_cache_seconds: 60
  structures_returned: 5
  max_learned_fill_bias_abs: 0.10
  max_learned_fill_bias_pct: 0.05
  min_calibration_fills_for_bias: 3

opportunity_weights_event:
  momentum: 0.15
  relative_volume: 0.15
  trend_regime: 0.10
  relative_strength: 0.10
  options_opportunity: 0.10
  options_liquidity: 0.10
  catalyst: 0.20
  corroboration: 0.05
  novelty: 0.05

opportunity_weights_momentum:
  momentum: 0.22
  relative_volume: 0.22
  trend_regime: 0.14
  relative_strength: 0.14
  options_opportunity: 0.14
  options_liquidity: 0.14
```

```
budget:
  per_session_ceiling_usd: 0.75
  daily_provider_ceiling_usd: 8.00
  openai_reserve_at_usd: 40.00
  openai_block_at_usd: 48.00
  anthropic_reserve_at_usd: 40.00
  anthropic_block_at_usd: 48.00
```

Keep provider model prices in configuration and verify them in the provider consoles before live use.

27. Demo Narrative

1. **Discovery:** show 200+ eligible symbols narrowing to 30, then 12, then 5. Highlight why the final symbol entered the pool: Core, fresh news/SEC, mover, most-active, or momentum.
2. **Track:** show whether it is an EVENT candidate or MOMENTUM candidate.
3. **Quant confirmation:** IEX price action, same-feed RVOL, relative strength and regime confirm the direction.
4. **Options pre-screen:** the engine retrieves real contracts and returns five two-leg debit spreads.
5. **Council:** Bull/Bear/Catalyst agents produce structured evidence; GPT PM proposes direction/thesis/risk.
6. **Claude:** Red Team PASS/MODIFY/VETO across evidence, thesis, data quality, structure, concentration and staleness/fidelity.
7. **Risk Constitution:** independently approves/resizes/rejects.
8. **Alpaca:** MCP submits the multi-leg paper limit order.
9. **Execution Quality:** show indicative adjusted reference vs actual fill and limit-walk behavior.
10. **Counterfactual Lab:** show GPT-original, Claude-modified and executed P&L plus selection/sizing attribution.
11. **Gate Lab:** show what rejected candidates would have done and sample sizes.

Closing line:

> Most AI trading agents tell you why they made a trade. Alpha Council shows how it found the opportunity, measures whether each decision layer added value, and proves what its risk gates saved or cost - including on trades it refused to take.

28. Scope Decisions and Deferred Features

28.1 Charles Schwab - deferred, optional post-MVP validator

The full Schwab read-only plane remains removed from the competition critical path. Claude's schedule reasoning was sound: cross-provider option normalization and OAuth handling can consume a build day and introduce operational failure.

v2.4 corrects one premise: Alpaca's Indicative feed should not be modeled as "every quote is 15 minutes old." Nonetheless, Schwab is still lower priority than discovery breadth, fill calibration, attribution and the dashboard.

Post-MVP MAY: after Alpha Council selects the exact final two-leg spread, a Schwab adapter may fetch only those two contracts and display a **warning-only** comparison. It MUST NOT:

- execute orders;
- block Alpaca execution;
- change risk limits automatically;
- become required for system startup;
- consume time before Counterfactual/Gate/Execution Quality features are complete.

28.2 Fed/BLS/BEA collectors - remain blackout/regime inputs

Keep scheduled macro events in `event_calendar.yaml`. Do not build three new catalyst collectors during the competition. Macro can influence market regime and blackouts without being treated as a single-name catalyst.

28.3 Investor-relations HTML scrapers - remain removed

SEC EDGAR + Alpaca News are the high-value/reliable sources. Bespoke issuer scrapers remain poor competition-week engineering economics.

28.4 Replay - capability retained via dependency injection
No separate replay subsystem is required. Re-run frozen decisions through the same orchestrator/provider interfaces.

28.5 Restored from v2.3 removal: opportunistic screeners
most-actives and movers return because they directly solve the static-universe blind spot. They are optional discovery sources only, protected by 403 session-disable behavior.

28.6 Still forbidden before MVP/dashboard completion
Next.js redesign, Postgres, social scraping, sentiment embeddings, vector databases, credit spreads, 0DTE, portfolio hedging, a second execution broker, reinforcement learning, fine-tuning, autonomous arbitrary web browsing, mobile app, Kubernetes.

29. Master Coding-Agent Instruction
Paste at the start of every Claude Code or Codex session:

You are implementing Alpha Council v2.4 from the attached implementation specification. The repository may already contain completed v2.3 work. Treat v2.4 as a patch-forward specification: preserve working components unless Section 31 explicitly requires a change.

Treat MUST/MUST NOT as normative. If an implementation conflict makes a requirement impossible, stop and explain the conflict before changing behavior.

Priority principle: SEARCH BROADER BEFORE LOWERING QUALITY.

For the next incomplete item:
1. inspect existing project code/tests and BUILD_STATUS.md;
2. identify whether the task is new work or a v2.3 -> v2.4 patch;
3. implement the smallest complete change satisfying v2.4;
4. add/update focused tests;
5. run relevant tests;
6. report files changed, tests run, measured API behavior, and unresolved issues;
7. stop before expanding scope unless asked.

Hard rules:
- Never expose or commit secrets.
- Never enable live Alpaca trading.
- Never replace deterministic risk rules with model prompts.
- Never let an LLM invent a contract, price, fill, account value, universe fact, or limit.
- Do not assume Indicative option quotes are universally 15 minutes old; use measured timestamps.
- Do treat Indicative prices as derived/non-OPRA estimates and calibrate against fills.
- Reject invalid bid/ask before midpoint calculation.
- Every rejection path writes exactly one gate_rejections row.
- Optional discovery sources must not become execution dependencies.
- Dynamic discovery is capped at 250 and never causes market-wide LLM scanning.
- Tier 3 must retain meaningful option liquidity.
- Never fabricate an API response shape; consult installed SDK/official docs.

30. Definition of Done
- ☐ Dedicated Alpaca competition paper account active, paper-locked, Level 3 options
- ☐ Monday RTH probe_data_reality.py complete; lag/drift config measurement-based

- ☐ Core ~65 always scanned
- ☐ Dynamic Discovery Universe works and is capped at 250
- ☐ Optional most-active/movers source probes and fails gracefully on 403
- ☐ Fresh Alpaca News can inject an eligible off-core symbol
- ☐ Funnel operates ~250 -> 30 -> 12 -> 5 -> <=3 normal Councils
- ☐ RTH-only 20-day IEX baselines; invalid quotes blocked
- ☐ EVENT and MOMENTUM tracks both work
- ☐ Options engine returns real two-leg debit verticals with Indicative-feed handling
- ☐ Tier 3 retains min volume >=5 and max spread <=22%
- ☐ GPT PM and Claude Red Team return validated structured objects
- ☐ Claude VETO/hard risk rules cannot be bypassed
- ☐ Breadth expansion occurs before Tier 2/Tier 3 relaxation
- ☐ Alpaca MCP executes idempotent multi-leg paper orders
- ☐ At least six complete order lifecycles demonstrated, without forcing six alpha bets
- ☐ Position monitor works with LLM providers offline
- ☐ Execution/Fill Calibration logs indicated-to-fill bias and limit-walk behavior
- ☐ Counterfactual ledger marks variants plus eligible rejected shadows; decomposition reconciles
- ☐ Gate Lab shows rejection counts/GateValue with sample sizes
- ☐ Discovery Funnel and Execution Quality appear in Streamlit dashboard
- ☐ council_once.py --decision-id reproduces frozen decisions including source/track/tier
- ☐ API spend visible and under $50 per provider
- ☐ All positions flat by competition flatten time
- ☐ README accurately describes IEX + Indicative feed constraints
- ☐ README, write-up, screenshots and video complete before submission

---

31. Mandatory v2.3 -> v2.4 Migration Patch

This section is written for the **current build status as of Aug 29**. Claude Code/Codex SHOULD execute these in order and update BUILD_STATUS.md after each checkpoint.

31.1 Do not rebuild these completed components

Preserve unless tests reveal defects:

- .env, .env.example, .gitignore
- settings.py
- db/schema.sql foundation and db/engine.py
- alpaca/rest_client.py rate limiting/pagination
- existing scoring.yaml, risk_constitution.yaml, universe.yaml as bases to patch
- probe_data_reality.py, init_db.py, API smoke tests

31.2 Patch configuration first

1. Rename config/document version to v2.4.
2. Keep the current ~65 symbols as core_symbols.
3. Add discovery and breadth_expansion settings from §26.
4. Replace Tier 3 values with v2.4 liquidity-preserving values.
5. Add EVENT/MOMENTUM weight sets and execution-calibration caps.

31.3 Build discovery before deep scanner

Implement:

- quant/discovery.py

- asset eligibility/cache with has_options
- TTL-based dynamic symbol state
- optional most-active and movers REST calls with 403 disable
- Stage-0 FastScore and top-30 selection
- source labels: CORE, ALPACA_NEWS, SEC_EVENT, MOST_ACTIVE, MOVER, OTHER_DYNAMIC

Tests MUST prove no LLM or option-chain request occurs at Stage 0.

31.4 Fix market data before scoring

- filter 20-day bars to RTH only;
- reject invalid bid/ask (ask <= 0, bid <= 0, ask < bid);
- backfill bars on-demand for injected symbols;
- re-run Monday quote-lag probe and update thresholds.

31.5 Implement track-aware scoring

- EVENT track uses catalyst/corroboration/novelty weights;
- MOMENTUM track reweights to technical/options components;
- both record candidate_track and universe_source to database/audit rows.

If the existing schema lacks these columns, add an additive migration rather than rebuilding the database.

31.6 Patch intelligence injection

- Alpaca News MAY inject off-core eligible symbols;
- SEC active-universe polling remains MUST;
- global/current-filings SEC injection is SHOULD and must fail gracefully if not robust.

31.7 Patch adaptive ladder

Replace pure time-based loosening with:

11:00 breadth expansion
12:30 Tier 2 if zero orders
14:00 breadth expansion again
14:15 Tier 3 if zero orders

Hard gates never move. Tier 3 liquidity retains volume >=5 and spread <=22%.

31.8 Add execution calibration before optional Schwab

Implement execution/fill_calibration.py and the dashboard metrics in §17.5. Log the first calibration spread Monday. Do not begin a Schwab adapter until:

- Council + risk + execution work end-to-end;
- counterfactual/gate attribution works;
- dashboard exists;
- execution calibration works.

31.9 Optional Schwab after MVP only

If all critical-path items are complete and OAuth is already reliable, implement a **final-spread warning-only validator** for the selected two contracts. Otherwise leave Schwab out of the competition build.

31.10 Update BUILD_STATUS.md

Add explicit rows for:

- dynamic discovery status;
- mover/most-active entitlement result;
- Monday Indicative-feed timestamp probe;
- EVENT/MOMENTUM tracks;
- execution-calibration fill count and median bias;

- dashboard freeze status.

---

32. External Implementation References (verify at coding time)

These references are included so Claude Code/Codex can confirm current API behavior instead of relying on model memory:

- Alpaca Market Data plan/coverage: https://docs.alpaca.markets/us/docs/about-market-data-api
- Alpaca Historical Option Data / Indicative feed description: https://docs.alpaca.markets/us/docs/historical-option-data
- Alpaca Most Active screener: https://docs.alpaca.markets/us/reference/mostactives-1
- Alpaca Market Movers screener: https://docs.alpaca.markets/us/v1.4.2/reference/movers-1
- Alpaca Assets (has_options attribute): https://docs.alpaca.markets/us/v1.4.2/reference/get-v2-assets-1
- Alpaca MCP server/tool inventory: https://docs.alpaca.markets/us/docs/alpaca-mcp-server

Implementation rule: if official docs or the live account response differ from this specification on an API response shape, **preserve the architectural intent but use the verified live/official shape** and document the discrepancy in BUILD_STATUS.md.

Alpha Council v2.4 - Dynamic-Discovery Edition. Supersedes v2.3 in full.